



Start with a declared goal, immutable data roles, and a transparent reference model.

Two scientific goals

Explanation / inference

- Estimate conditional associations
- Report coefficients and uncertainty
- Prioritize prespecified scientific terms

Prediction

- Measure performance out of sample
- Use repeated cross-validation
- Compare models on the same folds

Core objects and first functions

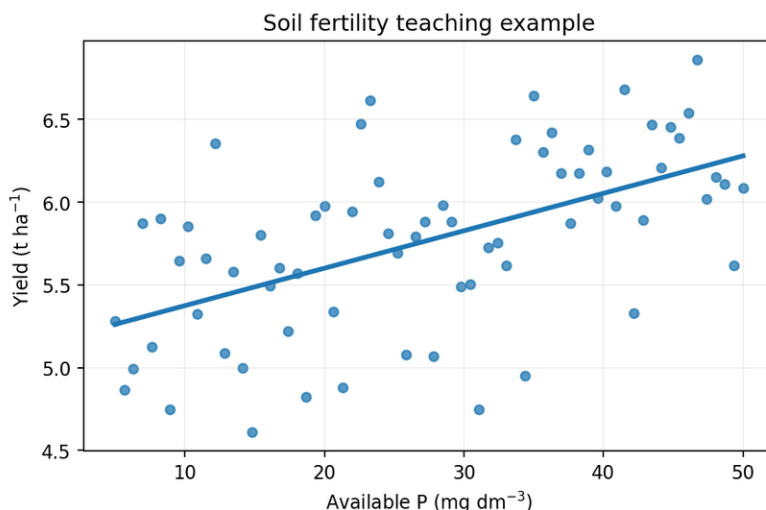
`mr_example_data()`

`mr_fit()`

`mr_capabilities()`

Five frozen synthetic datasets support reproducible teaching: `soil_fertility`, `plant_growth`, `agronomy_collinear`, `irrigation_nonlinear`, `agronomy_heteroskedastic`.

A first agronomic model



R workflow

```
soil <- mr_example_data("soil_fertility")
fit <- mr_fit(
  yield_t_ha ~ soil_pH + organic_matter_pct +
    available_P_mg_dm3 + season_rain_mm,
  soil, goal = "explanation")
mr_diagnose(fit)
```

Recommended first workflow

Define goal

Inspect data

Fit reference

Diagnose

Quantify uncertainty

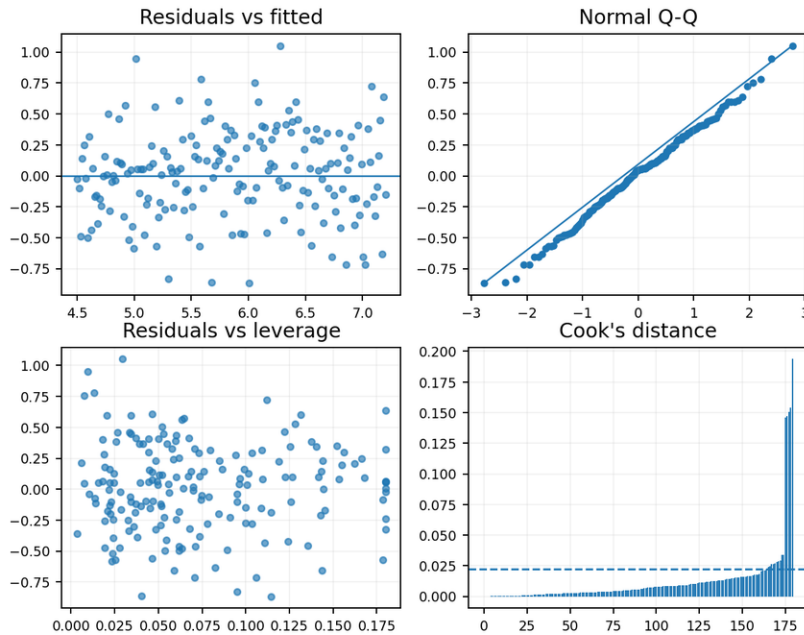
Interpret

The package intentionally separates inferential fit from predictive performance. Training R^2 is never used as a substitute for validation.



Diagnose assumptions and influential observations before changing the model.

A diagnostic panel



What `mr_diagnose()` checks

- Residual location and spread
- Shapiro screen for residual normality
- Breusch-Pagan heteroscedasticity screen
- Specification / nonlinearity screen
- Model rank and residual degrees of freedom
- Cook screening count

What `mr_influence()` returns

- Leverage
- Cook distance
- DFFITS
- DFBETAS
- Studentized residuals
- Transparent logical flags

Interpretation: flags are prompts, not deletion rules

- Inspect the biological and experimental reason for an unusual observation.
- Run sensitivity analysis with and without influential records only when scientifically defensible.
- A large Cook value does not automatically mean “bad data”.
- For designed experiments, trace plots, blocks, treatments, environments and measurement provenance.

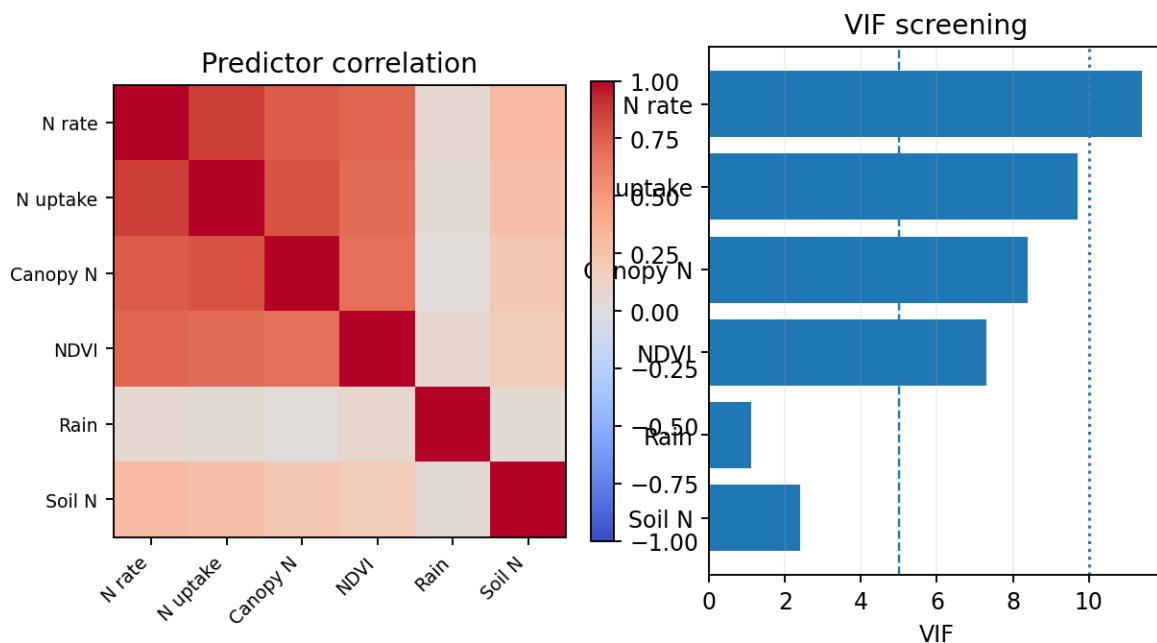
R workflow

```
diag <- mr_diagnose(fit)
infl <- mr_influence(fit)
subset(infl, flag_cook | flag_dffits | flag_dfbeta)
```



Measure redundancy, identify correlated predictor groups, and choose a scientifically defensible response.

Three complementary views



mr_collinearity()

- VIF and tolerance
- Condition indices from the predictor matrix
- Variance-decomposition proportions
- Optional performance::check_collinearity() output
- Flags at VIF 5 and 10 are descriptive screens

R workflow

```
coll <- mr_example_data("agronomy_collinear")
fit <- mr_fit(yield_t_ha ~ ., coll)
col <- mr_collinearity(fit)
col$VIF
col$condition
```

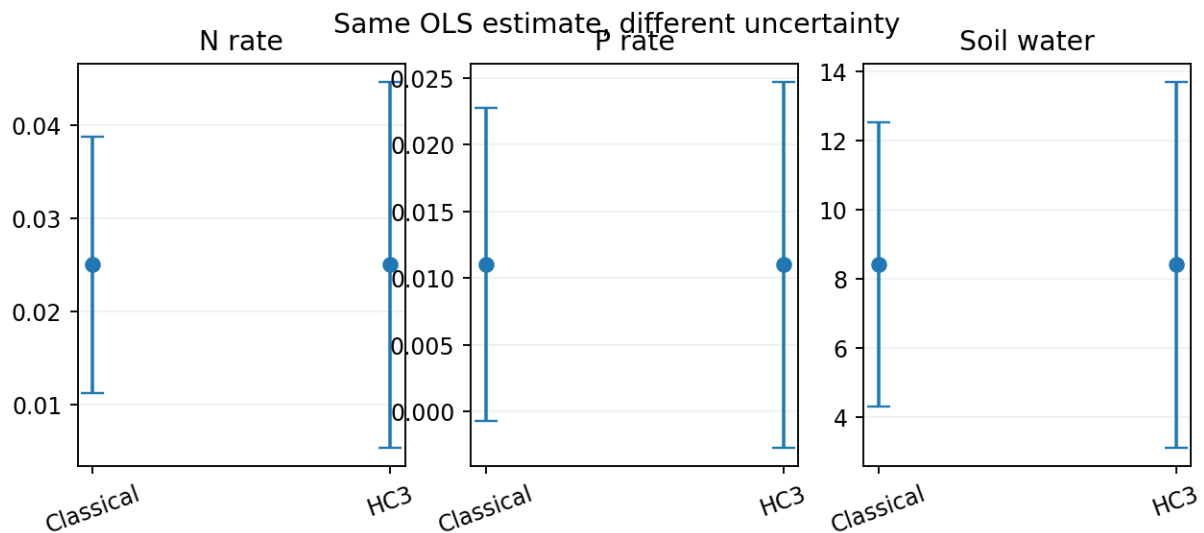
Decide with the science, not one threshold

- Keep a predictor when it represents a distinct agronomic construct.
- Combine variables only when the new construct is meaningful.
- Use ridge when prediction benefits from retaining correlated information.
- Use selection or stability tools when parsimony is the declared goal.
- Centering changes interpretation but does not remove structural redundancy.



Keep the OLS estimand while adapting standard errors to heteroscedasticity or clustering.

Classical vs heteroscedasticity-consistent uncertainty



mr_inference()

- classical
- HC0, HC1, HC2, HC3
- HC4, HC4m, HC5
- cluster

HC estimators preserve the OLS point estimates. What changes is the estimated covariance matrix, which changes standard errors, tests and confidence intervals.

R workflow

```
mr_inference(fit, "classical")
mr_inference(fit, "HC3")
mr_inference(fit, "HC4")
```

Cluster-robust field inference

- Use clusters for genuinely grouped sampling or experimental dependence.
- Examples: field, farm, block, site, season, or management unit.
- Cluster degrees of freedom are based on the number of clusters ($G - 1$).
- Few clusters require special caution and may need methods beyond the current package.

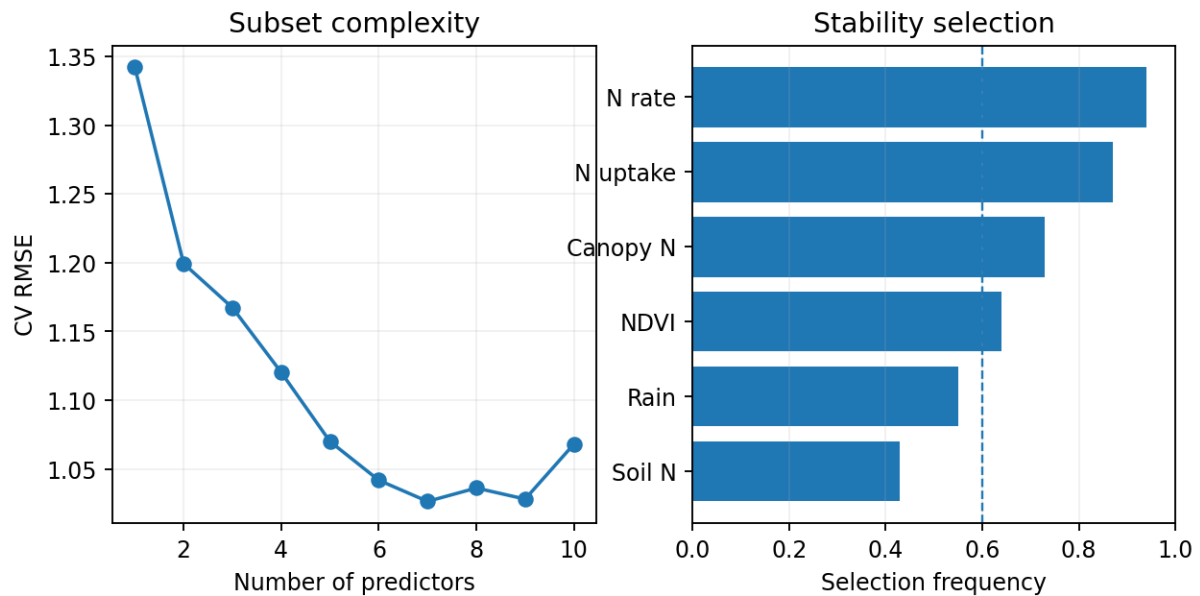
R workflow

```
mr_inference(fit, "cluster", cluster = "field")
mr_inference(fit, "cluster", cluster = ~ block)
```



Treat automated selection as exploratory and compare parsimony with stability.

Model complexity and selection frequency



mr_select()

- AIC stepwise
- BIC stepwise
- Best subsets
- Lasso / elastic net selection
- Stability selection via stabs
- keep = protects prespecified terms

R workflow

```
mr_select(fit, "aic")  
mr_select(fit, "bic", keep = "season_rain_mm")  
mr_select(fit, "best_subset")  
mr_select(fit, "stability", B = 100)
```

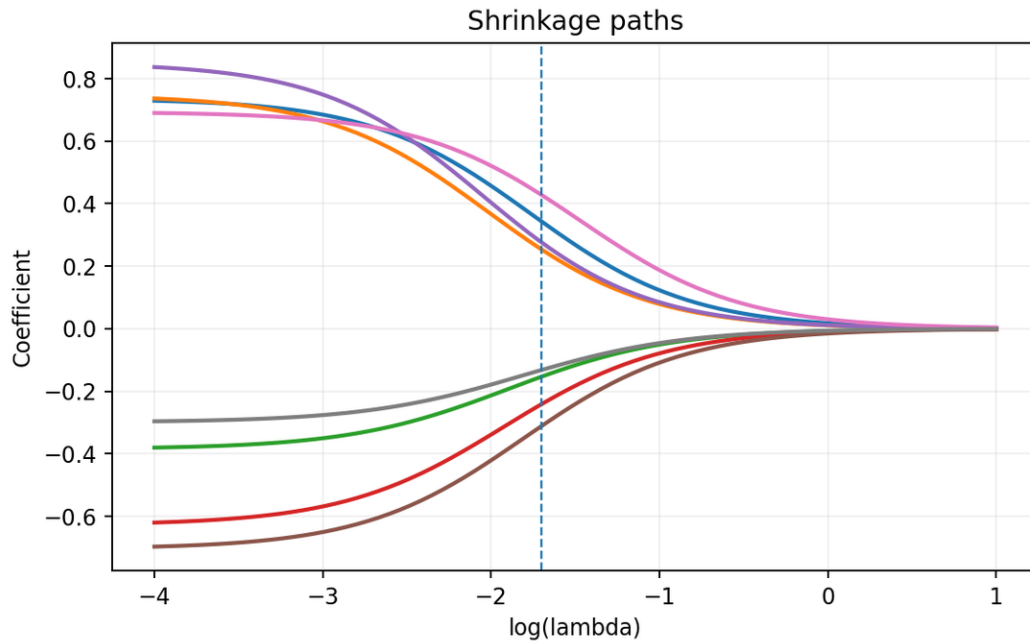
Good practice

- Start from a scientifically defensible candidate set.
- Do not interpret post-selection p-values as if the model had been prespecified.
- Use stability frequencies to reveal fragile choices.
- For prediction, evaluate the entire modeling process out of sample.
- Report what was selected, how, and under which tuning rule.



Shrink correlated predictors and control prediction variance with cross-validated tuning.

Coefficient paths as lambda changes



mr_regularize()

- ridge: $\alpha = 0$
- lasso: $\alpha = 1$
- elastic_net: $0 < \alpha < 1$
- lambda.1se favors a simpler solution
- lambda.min minimizes cross-validated error
- Returns coefficients and non-zero variables

R workflow

```
mr_regularize(fit, "ridge", nfolds = 10)
mr_regularize(fit, "lasso", lambda = "1se")
mr_regularize(fit, "elastic_net", alpha = 0.5)
```

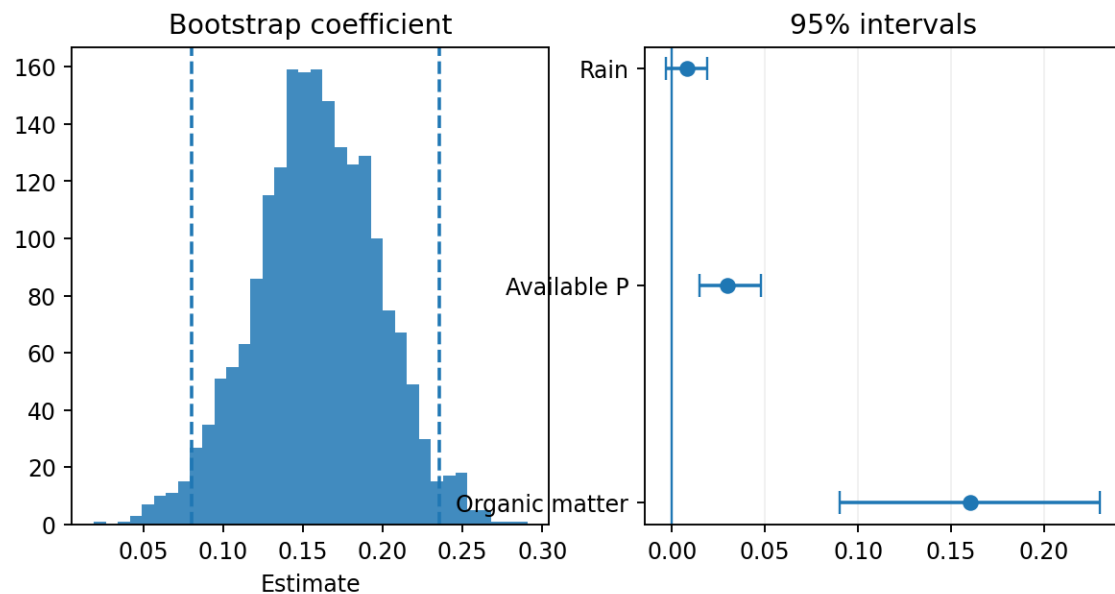
When it helps in agronomy

- N rate, N uptake, canopy N and NDVI often carry overlapping information.
- Ridge can retain all signals when prediction is the priority.
- Lasso yields sparse models but can be unstable under strong correlation.
- Elastic net balances sparsity and grouped retention.



Resample the data-generating structure to quantify coefficient uncertainty and sign stability.

Bootstrap distributions and intervals



mr_bootstrap()

- pairs bootstrap
- residual bootstrap
- wild bootstrap (Rademacher weights)
- percentile or basic intervals
- coefficient sign stability
- set a seed for reproducibility

R workflow

```
mr_bootstrap(fit, R = 2000, type = "pairs")  
mr_bootstrap(fit, R = 2000, type = "residual")  
mr_bootstrap(fit, R = 2000, type = "wild")
```

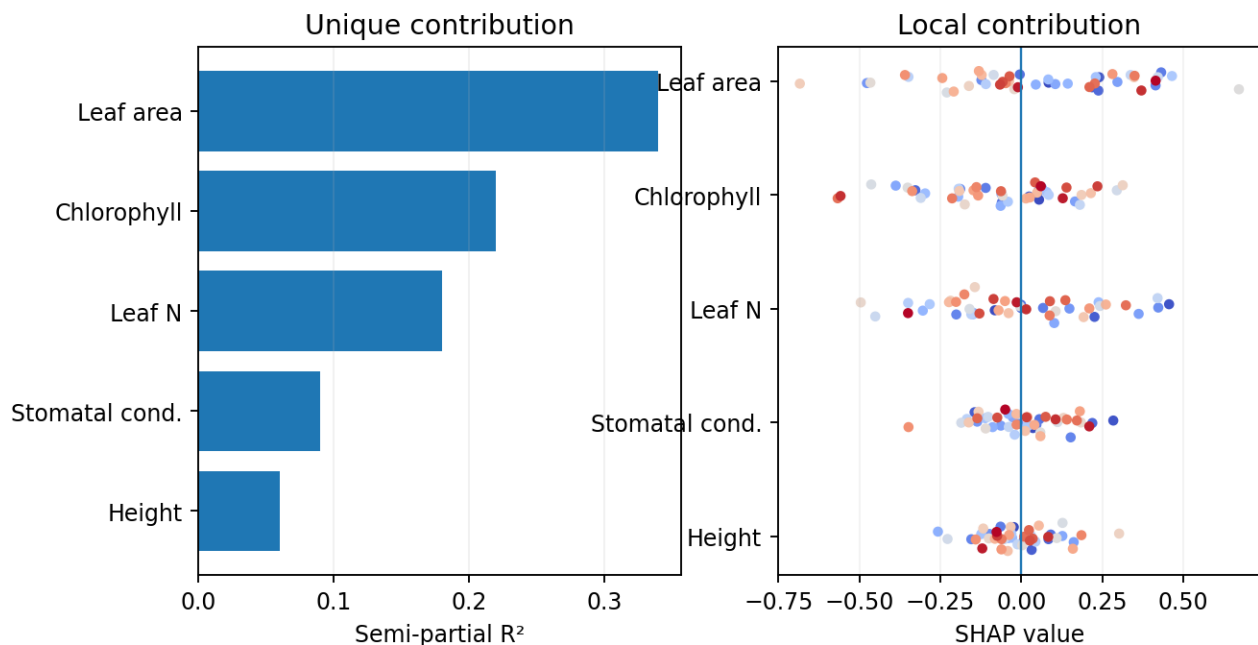
Match the bootstrap to the design

- Pairs resampling is often the safest generic choice for random-X regression.
- Residual bootstrap relies more strongly on the fitted error structure.
- Wild bootstrap is useful under heteroscedasticity.
- Clustered, hierarchical, spatial or repeated-measures designs may need specialized resampling beyond this interface.



Use complementary importance definitions instead of forcing one universal ranking.

Global and local contributions



mr_importance()

- standardized beta
- partial R^2
- semi-partial R^2
- LMG relative importance
- additive SHAP for lm

Partial R^2 answers how much a predictor adds relative to the remaining unexplained variance. Semi-partial R^2 expresses unique contribution to total outcome variance.

R workflow

```
mr_importance(fit, "semi_partial_r2")
mr_importance(fit, "lmg")
mr_importance(fit, "shap")
```

mr_effects()

- Average marginal slopes via margineffects
- Useful for transformed or flexible model interpretation
- Report effect size with uncertainty, not only importance
- Keep causal language separate from predictive explanation

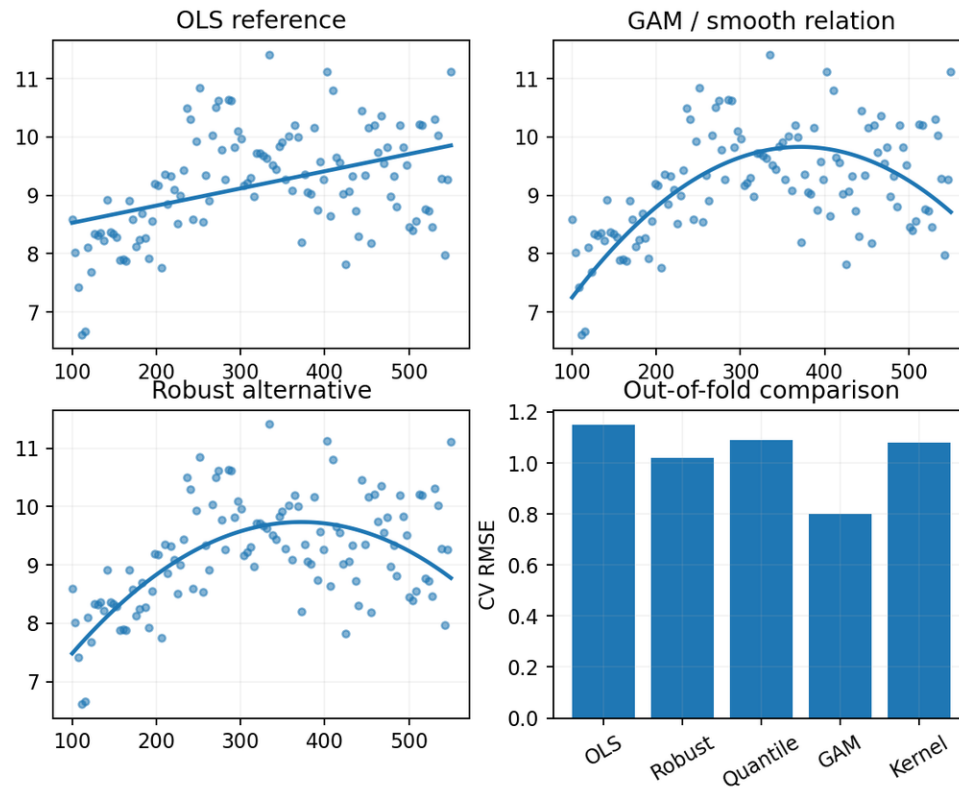
R workflow

```
mr_effects(fit)
mr_effects(gam_fit)
```



Leave OLS when the scientific signal or error structure demands it, then compare honestly out of sample.

OLS, robust, quantile, GAM, kernel - then cross-validate



Alternative engines

- robust: resistant regression
- quantile: conditional median or other tau
- gam: smooth nonlinear effects
- kernel: nonparametric regression

R workflow

```
mr_fit(..., engine = "robust")
mr_fit(..., engine = "quantile", tau = 0.5)
mr_fit(..., engine = "gam")
mr_fit(..., engine = "kernel")
```

mr_validate() and mr_compare()

- Repeated v-fold out-of-fold prediction
- RMSE, MAE and predictive R^2
- Calibration intercept and slope
- Compare multiple models on aligned folds
- Inference and prediction remain separate tables

R workflow

```
mr_validate(fit, v = 10, repeats = 5, seed = 123)
mr_compare(ols = fit, gam = gam_fit, v = 10)
```



Turn a regression analysis into an auditable sequence of decisions, diagnostics, validation, and deliverables.

End-to-end multiRegFlow

- | | |
|---|--|
| 1 Data
<code>mr_example_data()</code> | 6 Improve
<code>mr_select()</code> / <code>mr_regularize()</code> |
| 2 Fit
<code>mr_fit()</code> | 7 Uncertainty
<code>mr_bootstrap()</code> |
| 3 Diagnose
<code>mr_diagnose()</code> | 8 Interpret
<code>mr_importance()</code> / <code>mr_effects()</code> |
| 4 Collinearity
<code>mr_collinearity()</code> | 9 Validate
<code>mr_validate()</code> / <code>mr_compare()</code> |
| 5 Infer
<code>mr_inference()</code> | 10 Report
<code>mr_recommend()</code> / <code>mr_report()</code> |

Scientific record

- Declared goal
- Formula and engine
- Diagnostic findings
- Inference method
- Selection/tuning choices
- Seeds and resampling settings

Decision support

- `mr_recommend()` summarizes detected issues
- Recommendations remain transparent
- No automatic variable deletion
- No training R^2 as predictive evidence

Deliverables

- `mr_report()` Markdown output
- Tables for coefficients and diagnostics
- Plots for communication
- Validation metrics
- Reproducible session information

R workflow

```
soil <- mr_example_data("soil_fertility")
fit <- mr_fit(yield_t_ha ~ ., soil, goal = "explanation")
mr_diagnose(fit); mr_collinearity(fit)
mr_inference(fit, "HC3")
mr_importance(fit, "semi_partial_r2")
mr_recommend(fit)
mr_report(fit, file = "soil-regression-report.md")
```